

Digital Design and Computer Organisation Laboratory

UE23CS251A

3rd Semester, Academic Year 2024-25

DDCO HACKATHON

Date:20/11/2024

Name:	SRN:	Section
Dipak Rajendra Chaudhari	PES1UG24CS804	K
Vaibhav Srivastava	PESS1UG23CS666	K
Tejas Anil Nagmote	PES1UG23CS646	K
Utkarsh Vats	PES1UG23CS661	K

TITLE:

1. Design a iverilog code for ALU, register file and concatenate Alu –Reg for
 - 8 bit processor which will take 2 bit op code to perform only logical (and ,or, nand ,nor) operation
 - Memory should have 8 register of each size of eight bit and save the result in one of the register and perform read operation to know the output.

Register file.v:-

```
module register_file (  
    input clk,  
    input [2:0] write_address,  
    input [7:0] write_data,  
    input write_enable,  
    input [2:0] read_address,  
    output reg [7:0] read_data  
);  
reg [7:0] registers [0:7]; // 8 registers of 8-bit size  
  
always @(posedge clk) begin  
    if (write_enable) begin  
        registers[write_address] <= write_data; // Write data to register
```

```

        end
    end

    always @(*) begin
        read_data = registers[read_address]; // Read data from register
    end
endmodule

```

alu_register_tb.v

```

`timescale 1ns/1ps

module alu_register_tb;

    reg clk;

    reg [7:0] a, b;

    reg [1:0] opcode;

    reg [2:0] write_address, read_address;

    reg write_enable;

    wire [7:0] read_data;

    // Instantiate the top module
    alu_register_concat dut (

        .clk(clk),

        .a(a),

        .b(b),

        .opcode(opcode),

        .write_address(write_address),

        .read_address(read_address),

        .write_enable(write_enable),

        .read_data(read_data)

    );

```

```

// Clock generation

initial begin

    clk = 0;

    forever #5 clk = ~clk; // Generate clock with a period of 10
units

end

initial begin

    // Test cases

    $dumpfile("alu_register.vcd"); // VCD file generation

    $dumpvars(0, alu_register_tb);

    // Initialize inputs

    a = 8'b10101010;

    b = 8'b11001100;

    opcode = 2'b00; // AND operation

    write_address = 3'b000;

    write_enable = 1;

    #10; // Wait for 10 time units

    write_enable = 0; // Disable writing

    opcode = 2'b01; // OR operation

    write_address = 3'b001;

    write_enable = 1;

    #10;

    write_enable = 0;

    opcode = 2'b10; // NAND operation

    write_address = 3'b010;

```

```

write_enable = 1;

#10;

write_enable = 0;

opcode = 2'b11; // NOR operation

write_address = 3'b011;

write_enable = 1;

#10;

write_enable = 0;

// Read operation

read_address = 3'b000; #5;

$display("Read Data (AND): %b", read_data);

read_address = 3'b001; #5;

$display("Read Data (OR): %b", read_data);

read_address = 3'b010; #5;

$display("Read Data (NAND): %b", read_data);

read_address = 3'b011; #5;

$display("Read Data (NOR): %b", read_data);

$finish; // End simulation

end

endmodule

```

alu_register_concat.v:-

```

module alu_register_concat (
    input clk,
    input [7:0] a,
    input [7:0] b,

```

```

input [1:0] opcode,
input [2:0] write_address,
input [2:0] read_address,
input write_enable,
output [7:0] read_data
);
wire [7:0] alu_result;
// Instantiate ALU
alu alu_inst (
    .a(a),
    .b(b),
    .opcode(opcode),
    .result(alu_result)
);
// Instantiate Register File
register_file register_file_inst (
    .clk(clk),
    .write_address(write_address),
    .write_data(alu_result),
    .write_enable(write_enable),
    .read_address(read_address),
    .read_data(read_data)
);
endmodule

```

alu_register_tb.v :-

```

`timescale 1ns/1ps
module alu_register_tb;
    reg clk;
    reg [7:0] a, b;
    reg [1:0] opcode;
    reg [2:0] write_address, read_address;

```

```

reg write_enable;
wire [7:0] read_data;
// Instantiate the top module
alu_register_concat dut (
    .clk(clk),
    .a(a),
    .b(b),
    .opcode(opcode),
    .write_address(write_address),
    .read_address(read_address),
    .write_enable(write_enable),
    .read_data(read_data)
);
// Clock generation
initial begin
    clk = 0;
    forever #5 clk = ~clk; // Generate clock with a period of 10 units
end
initial begin
    // Test cases
    $dumpfile("alu_register.vcd"); // VCD file generation
    $dumpvars(0, alu_register_tb);
    // Initialize inputs
    a = 8'b10101010;
    b = 8'b11001100;
    opcode = 2'b00; // AND operation
    write_address = 3'b000;
    write_enable = 1;
    #10; // Wait for 10 time units
    write_enable = 0; // Disable writing
    opcode = 2'b01; // OR operation
    write_address = 3'b001;
    write_enable = 1;

```

```

#10;
write_enable = 0;
opcode = 2'b10; // NAND operation
write_address = 3'b010;
write_enable = 1;
#10;
write_enable = 0;
opcode = 2'b11; // NOR operation
write_address = 3'b011;
write_enable = 1;
#10;
write_enable = 0;
// Read operation
read_address = 3'b000; #5;
$display("Read Data (AND): %b", read_data);
read_address = 3'b001; #5;
$display("Read Data (OR): %b", read_data);
read_address = 3'b010; #5;
$display("Read Data (NAND): %b", read_data);
read_address = 3'b011; #5;
$display("Read Data (NOR): %b", read_data);
$finish; // End simulation
end
endmodule

```

Alu.v:-

```

module alu (
input [7:0] a,
input [7:0] b,
input [1:0] opcode,
output reg [7:0] result
);
always @(*) begin
    case (opcode)

```

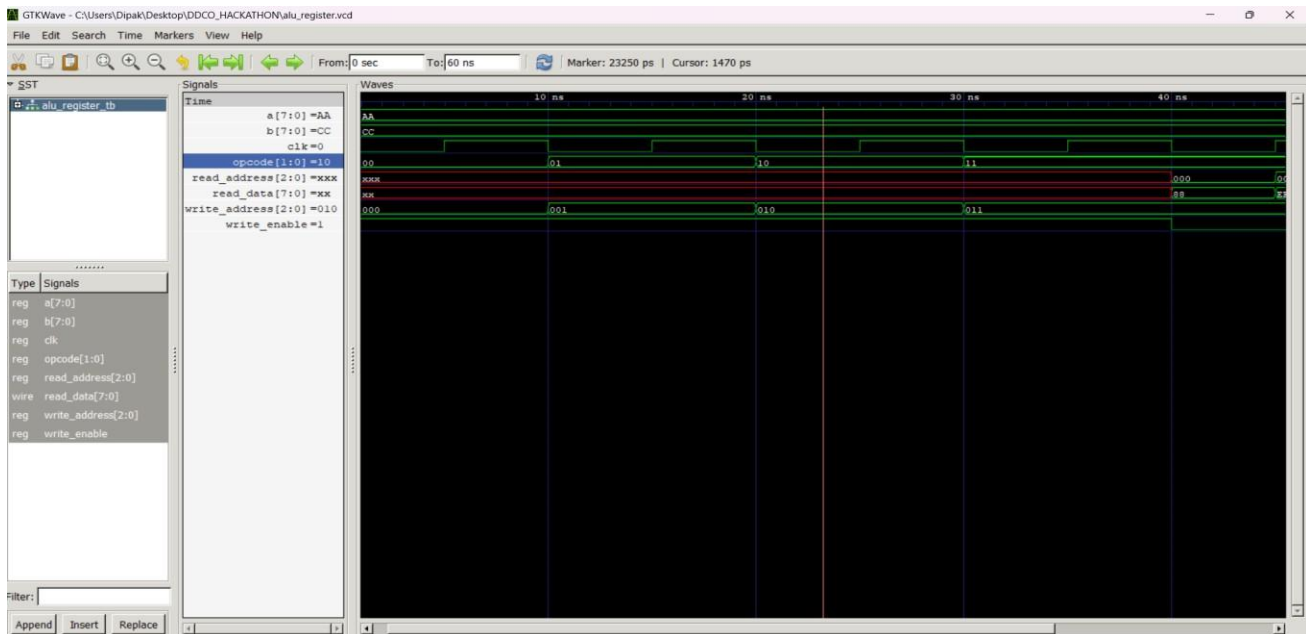
```
2'b00: result = a & b; // AND
2'b01: result = a | b; // OR
2'b10: result = ~(a & b); // NAND
2'b11: result = ~(a | b); // NOR
default: result = 8'b00000000; // Default case
endcase
end
endmodule
```

Outputs:-

Console Output:-

```
PS C:\Users\Dipak\Desktop\DDCO_HACKATHON> vvp alu_register
VCD info: dumpfile alu_register.vcd opened for output.
Read Data (AND): 10001000
Read Data (OR): 11101110
Read Data (NAND): 01110111
Read Data (NOR): 00010001
alu_register_tb.v:78: $finish called at 60000 (1ps)
```

GTKWAVE OUTPUT:-



Originality Statement:-

This project is the result of our original ideas and collaborative effort during the hackathon. All the work presented here is entirely our own, developed without referencing or copying from any external sources.